

KODIGO

Kodigo System Design and Governance

Technical design and accountable governance for the Kodigo capstone system

Table 1. Document Control

Control	Value
Version and date	Version 1.2 5 October 2026
Status	Revised for peer review; formal organizational approval is pending.
Document owner	[REQUIRES PROJECT TEAM CONFIRMATION: identify accountable deploying organization and document owner]
Approving authority	[REQUIRES PROJECT TEAM CONFIRMATION: identify authorized approver and approval date]
Review cycle	Review at least annually and after material system, role, privacy, or workflow changes.
Evidence basis	Repository implementation and supporting project documents reviewed on 5 October 2026; deployment state is not inferred from source files.

Project-team confirmation fields are intentionally retained where repository evidence is insufficient.

Contents

Contents 2

Executive summary 3

PART I - SYSTEM DESIGN..... 3

1. Purpose, business need, scope, and stakeholders 3

2. Proposed measurable acceptance criteria 3

3. Architecture and major components 4

4. Technology choices and rationale 4

5. Major modules and functional requirements 5

6. Data model and relationships 5

7. Data flow and business process 6

8. Role and permission design..... 6

9. Security architecture and risk controls 7

10. Integrations, environments, and service identities 7

PART II - SYSTEM GOVERNANCE 8

11. Governance objectives and structure 8

12. Decision rights and segregation of duties 9

13. Access governance lifecycle..... 9

14. Data and privacy governance..... 9

15. Risk management 10

16. Change, release, and configuration management 10

17. Incident management and escalation 11

18. Backup, recovery, audit, and monitoring responsibilities 13

19. User feedback and post-launch improvement..... 13

20. Documentation ownership, training, and review 13

21. Feasibility, schedule, and resource plan 14

22. Course concept integration and evidence 15

23. Rubric traceability..... 15

24. Decisions requested before submission or production use 15

Executive summary

Kodigo is a multi-store retail operations system for sari-sari stores and small branches. It combines browser-based checkout, catalog and stock controls, supplier and purchase-order workflows, reporting, and store-level access management. The design uses a React and TypeScript frontend with Supabase Auth and Postgres, database-enforced store scoping, server-side functions for privileged administration, and browser storage for eligible offline work. This document separates technical design from governance so reviewers can see both how the system works and who must control it.

The repository provides substantial implementation evidence, but it does not prove that every migration is applied, that production hosting or backups are active, or that organization-specific policy owners and budgets are approved. Those items are flagged for team confirmation.

PART I - SYSTEM DESIGN

1. Purpose, business need, scope, and stakeholders

Small retailers need checkout, stock records, supplier coordination, and simple performance reporting to stay aligned across daily work. When sales and inventory records are separated, cashiers may repeat a payment, staff may reorder from stale stock, and owners may be unable to trace a correction. Kodigo addresses this operational need by linking sale records to product stock, preserving transaction lifecycle events, showing stock alerts and restocking information, and summarizing activity by store.

Table 2. System purpose and boundaries

Item	Description
Purpose	Support checkout, inventory accuracy, supplier procurement, and store-level review for small retail operations.
Primary users	Admin, Cashier, Inventory, Super Admin; technical maintainers and reviewers have separate organizational duties.
In scope	Browser application, Supabase Auth, Postgres data and policies, database functions and triggers, Edge Functions, IndexedDB offline queues, reports, and optional cash-drawer integration.
Out of scope unless separately confirmed	Card processing, storing full payment credentials, a mobile device management service, automated legal/tax certification, a named production service desk, or verified production hosting and backups.

2. Proposed measurable acceptance criteria

The following are proposed capstone acceptance tests, not reported test results. The project team should approve test data, test owner, environment, and completion dates before using them as final success claims.

Table 3. Proposed success measures

Measure	Target for capstone acceptance	Evidence to retain
Role and store boundaries	All agreed route and database access tests pass for Admin, Cashier, Inventory, and Super Admin; every unauthorized cross-store read/write attempt is denied.	Dated test matrix and sanitized results.
POS correctness	At least 20 representative checkout cases produce the expected totals, receipt references, and stock effects; zero duplicate sale records in the tested cases.	Staging test cases, transaction IDs, and stock reconciliation.
Offline recovery	Five controlled offline sales replay once after reconnection; every test sale is either synchronized or clearly retained as an error for investigation.	Queue status evidence and reconciliation log.
Inventory and purchasing	At least 10 product, adjustment, receiving, archive, and supplier scenarios match expected base-unit balances and history.	Scenario checklist and movement records.

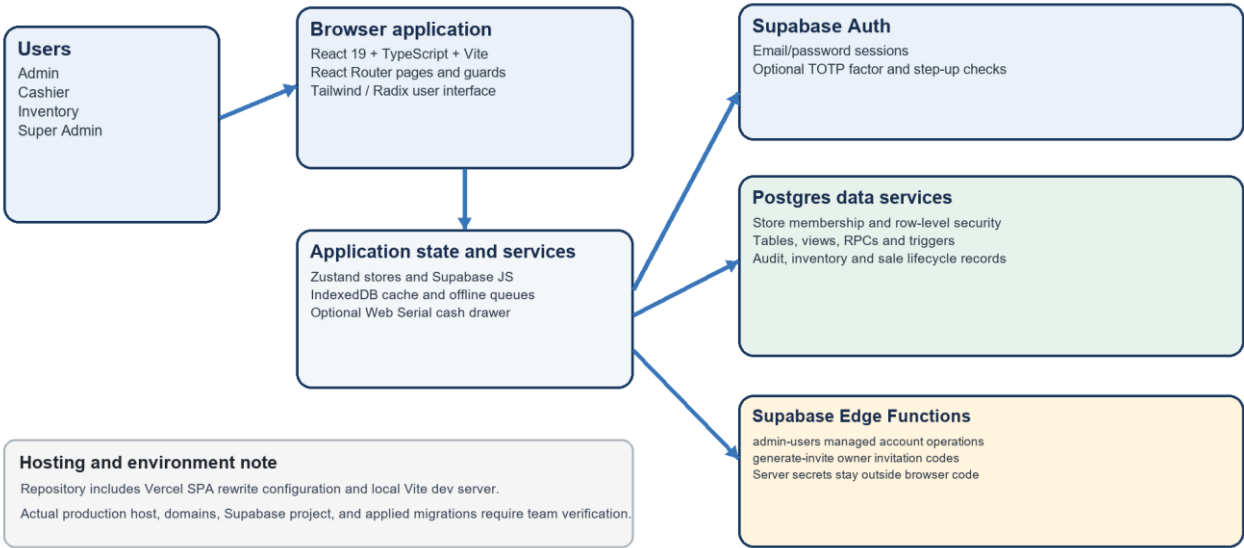
Measure	Target for capstone acceptance	Evidence to retain
Recovery readiness	Complete one restore rehearsal in an isolated project and reconcile stores, products, sales, sale items, and adjustments before release approval.	Restore log, row-count reconciliation, and approver sign-off.
User readiness	[REQUIRES PROJECT TEAM CONFIRMATION: set number of representative users, training scenarios, and pass threshold].	Attendance, scenario results, trainer sign-off.

3. Architecture and major components

The browser presents route-level screens and controls. Zustand stores manage session, active store, products, suppliers, cart, and alerts. The Supabase JavaScript client connects authenticated users to the backend. Supabase Auth issues sessions; Postgres policies constrain data; database RPCs and triggers carry business operations; Edge Functions perform privileged user and invite actions. IndexedDB holds a product cache and eligible offline queues. Web Serial is optional and limited to a compatible cash-drawer command.

Figure 1. Kodigo Logical Architecture

Kodigo current logical architecture



4. Technology choices and rationale

Table 4. Technology stack and design reason

Layer	Current implementation	Reason for use / constraint
Frontend	React 19, TypeScript 5.9, Vite 7	Component-based interactive UI, typed application contracts, fast local development/build. TypeScript does not replace server authorization.
Navigation and state	React Router 7, Zustand 5	Route organization and small focused stores for authentication, product, supplier, cart, and alert state.
UI and charts	Tailwind CSS 4, Radix UI primitives, Lucide React, Recharts 3	Consistent responsive controls, accessible UI primitives, and interactive business charts.
Backend	SupabaseAuth, Postgres, row-level security, RPCs, triggers, views, Edge Functions	Managed sign-in and relational storage; database policies and functions enforce rules close to data. Actual target project and plan require confirmation.

Layer	Current implementation	Reason for use / constraint
Offline	IndexedDB through idb	Browser-local product cache and queues support eligible work during connectivity interruptions; conflict resolution is limited and queues require careful reconciliation.
Hardware and exports	Web Serial for optional ESC/POS drawer command; jsPDF and write-excel-file for exports	Uses browser capabilities for optional peripherals and report output. Printer/drawer compatibility must be tested.
Hosting	Vercel SPA rewrite configuration exists in repository	Actual deployed host, domain, production project, CI release permissions, and hosting cost: [REQUIRES PROJECT TEAM CONFIRMATION: confirm deployed configuration and owners].

5. Major modules and functional requirements

Table 5. Implemented module requirements

ID / module	Current behavior supported by repository	Expected result
FR-01 Identity and store context	Email/password sign-in, invite-based owner registration, role resolution, assigned store loading, and role-based route routing.	A user sees only authorized screens and store data.
FR-02 POS and receipts	Search by name/SKU/barcode, cart, unit/bulk options, payment, receipt snapshot, and sales lifecycle actions through database functions.	Sale, sale items, payment details, and stock effects are recorded consistently.
FR-03 Offline continuity	Cache products; queue eligible sales and selected mutations in IndexedDB; attempt replay on reconnect or page activity.	Pending/error state remains visible for reconciliation; business rule errors are not silently treated as network failures.
FR-04 Product and inventory	Product/category maintenance, purchase and selling units, price history, stock adjustment and movement records, archive/restore, and consumption history.	Base stock and its history remain traceable; sold products are archived rather than deleted.
FR-05 Suppliers and purchasing	Supplier records and store links, product supplier assignments, restocking options, purchase orders, receiving functions, supplier scoring, and consignment lots/balances.	Delivery and supplier-owned stock can be distinguished and reviewed.
FR-06 Alerts and notifications	Stock and selling-option conditions create notifications with user read state; report-export and transaction events can be surfaced.	Authorized users can review operational notices for assigned context.
FR-07 Reporting and analytics	Sales and inventory reports, Excel/PDF export paths, revenue/profit/category/hour charts, transaction views, rankings, and velocity.	Users review filtered output and export only within their role.
FR-08 Administration	Store branding and settings, managed staff account operations through admin-users Edge Function, invite generation through generate-invite, password reset, TOTP management.	Authorized changes are attributed and limited to the required store scope.

6. Data model and relationships

The schema is relational and store-oriented. Most operational records carry a store identifier or reach a store through an approved mapping. Row-level security (RLS) means database policies decide which rows an authenticated user can read or change for each request.

Table 6. Core data entities

Domain	Entities evidenced in schema and migrations	Relationship / purpose
Identity and tenancy	`profiles`, `stores`, `store_users`, `invite_codes`	Profiles map to stores through store_users; role and membership scope access.
Catalog and stock	`categories`, `products`, `product_selling_options`, `product_restocking_options`, `stock_adjustments`, `inventory_movements`, `inventory_stock_lots`	Products belong to store/category; stock movements explain sales, receipt, loss, adjustment, and consignment allocation.

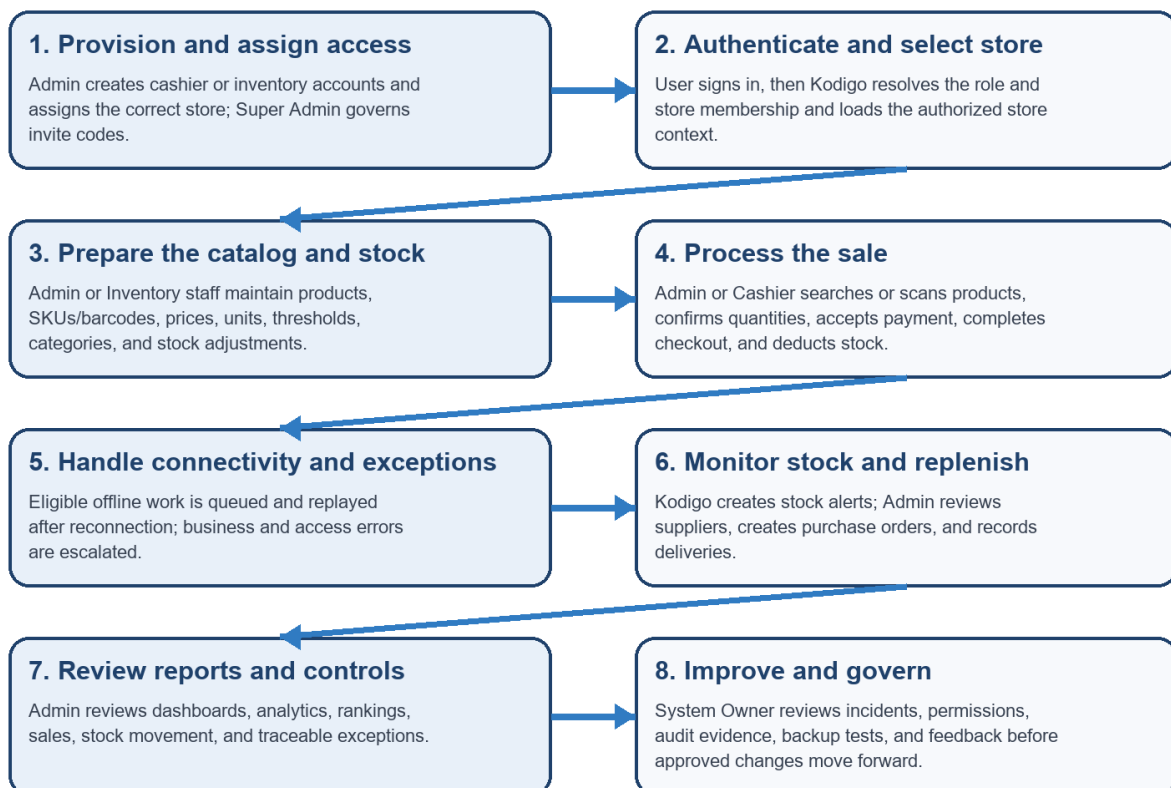
Domain	Entities evidenced in schema and migrations	Relationship / purpose
Sales	`sales`, `sale_items`, `sale_payments`, `receipts`, `sale_events`, `sale_returns`, `sale_return_items`, `cashier_closeouts`	A sale has line items, payment and receipt records, with lifecycle events for void/refund/return and closeout.
Procurement	`suppliers`, `supplier_stores`, `product_suppliers`, `purchase_orders`, `purchase_order_items`	Supplier-store links support shared suppliers; orders link suppliers and products to receiving events.
Operations and evidence	`notifications`, `notification_user_states`, `audit_logs`, `error_logs`, `receipt_reprints`, `product_price_history`	Operational notices, user read status, errors, audit evidence, reprints, and price history support monitoring and traceability.

The list reflects repository migrations and application queries, not a verified live database inventory. The target project's applied migration ledger and actual grants must be inspected before release.

7. Data flow and business process

A named user signs in, receives a role and authorized store context, and performs an approved task. An online POS sale goes through the sale-processing RPC, which creates sale and item records and applies inventory effects. During an eligible network interruption, the browser may queue work locally and replay it later. Store staff reconcile any error or duplicate risk before closing the shift. Inventory movement, supplier delivery, alerts, reports, and review complete the operating cycle.

Figure 2. Proposed Kodigo Business Process Flow



8. Role and permission design

Table 7. Application permission matrix

Capability	Admin	Cashier	Inventory	Super Admin
------------	-------	---------	-----------	-------------

Capability	Admin	Cashier	Inventory	Super Admin
POS checkout	Execute where enabled	Execute	None	None
Products and stock	Full store control	Read at POS	Manage within assigned scope	None
Stock adjustment / receiving	Manage and approve per local controls	None	Record within role and store scope	None
Suppliers and purchase orders	Manage assigned stores	None	No supplier administration	None
Dashboard / analytics / rankings	Assigned-store view	None	Limited per route	None
Reports	Sales and inventory	None	Inventory-oriented reports	None
Store settings / staff management	Manage assigned stores via supported controls	Own account	Own account	None
Invite governance	None	None	None	Generate owner invitation codes
Account security	Own account	Own account	Own account	Own account

This summarizes implemented route intent and code paths. Database policies are the authority; route guards improve navigation but cannot replace backend enforcement. The exact live permission matrix must be validated against the deployed migration state.

9. Security architecture and risk controls

Table 8. Security controls classified by purpose

Control class	Control evidenced or required	Risk addressed / evidence
Preventive	Supabase Auth sessions; role checks; store membership; row-level security; authorized RPC checks.	Blocks unauthorized access and cross-store operations; validate with role/store test cases.
Preventive	TOTP enrollment and assurance checks for protected operations where enabled; password reset flow.	Reduces account takeover; role-wide MFA requirement and target project settings require confirmation.
Preventive	Server-side admin-users and generate-invite functions; service-role secrets stay in server environment.	Limits privileged user-management operations; verify only required secrets are configured and never expose them in VITE variables.
Preventive	Store-specific active context for POS; database sale/receive/adjustment RPCs; validation and lifecycle rules.	Reduces wrong-store posting and direct stock mutation.
Detective	`audit_logs`, `sale_events`, `inventory_movements`, `error_logs`, closeouts, price history, and export notifications.	Supports accountability and incident reconstruction; retention, access review, and alert ownership must be assigned.
Detective	Quarterly role/store review and release/test evidence.	Detects stale or excessive privileges and unsafe changes; retain dated approval records.
Corrective	Revoke or change access, reset credentials, correct through traceable adjustment/return functions, and preserve error queues.	Contains compromise and repairs records without concealing prior events.
Corrective	Production backup and restore procedure is described in the operations runbook.	Recovery capability depends on a configured backup plan and a successful restore drill; actual state must be verified.

10. Integrations, environments, and service identities

Table 9. Environment and identity inventory

Area	Repository evidence	Access / status	Confirmation required
Local development	Vite dev server in `kodigo-ui/`; `.env.example` names public Supabase URL and anon/publishable key.	Developer workstation; do not use production data for routine tests.	Document approved local data and credential handling.

Area	Repository evidence	Access / status	Confirmation required
Test / staging	Integration harness expects an isolated project and test credentials; CI workflow is present.	Use only test users and non-production data; service-role key only in protected test environment.	Confirm project ID, owner, migration ledger, access list, and retention.
Production	Application supports Supabase integration; Vercel SPA rewrite config is present.	Deployment existence, domain, plan, and applied migration version are not verifiable from repository files.	[REQUIRES PROJECT TEAM CONFIRMATION: name production host/project/domain, owner, region, and applied migration list].
Browser client	VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY in example configuration.	Public client key may be delivered to browser; database policies must limit access.	Confirm only public keys are present in VITE build variables.
Edge Function runtime	`admin-users` and `generate-invite` functions; Supabase server environment includes service-role secret by design.	No human login sharing; secret remains in function environment and should be rotated under provider controls.	[REQUIRES PROJECT TEAM CONFIRMATION: identify deployed function identities, secret owner, rotation schedule, and emergency access approver].
Backup identity	Operations runbook recommends restricted backup storage and restore roles.	Actual backup subscription, vault, service account, and retention not verified.	[REQUIRES PROJECT TEAM CONFIRMATION: name backup operator, restricted storage location, retention, and restore approver].

Migration caution: the repository contains numbered migrations through migration_40 and additional dated migrations. The README setup sequence does not include every newest file. Treat the applied migration ledger in the target Supabase project as the deployment source of truth; do not assume repository files are already deployed or invent an execution order where the environment is unknown.

PART II - SYSTEM GOVERNANCE

11. Governance objectives and structure

Governance assigns decision rights for access, changes, data, security, incidents, backups, and documentation. The deploying organization owns the service and accepts operational and privacy obligations. The roles below are governance assignments; only Admin, Cashier, Inventory, and Super Admin are application roles in the current TypeScript role union.

Table 10. Governance responsibilities

Governance role	Owns / maintains	Approves / decides	Accountability
System Owner / deploying organization	Business purpose, user outcomes, service scope, policy and resource approval.	Accepts business changes, assigns owners, accepts residual risk.	Business impact, support capacity, and service direction.
Admin / store owner	Store configuration, product and staff accuracy, local access, daily reconciliation.	Routine account/store access, stock corrections, and purchase actions under local thresholds.	Store operation and correct, timely escalation.
Cashier	Accurate cart, payment, receipt, and shift handoff.	No role authority to approve own exceptions beyond assigned actions.	Actions performed under own credentials; prompt discrepancy report.
Inventory staff	Catalog, physical counts, movement records, and inventory reports.	Routine stock activity within role; Admin approves unusual changes per policy.	Count accuracy, conversion correctness, and evidence notes.
Super Admin	Owner invite-code governance.	Issue governed owner invitation codes.	Invite issuance and onboarding integrity; no tenant operations.
System Administrator	Supabase configuration, technical access, releases, logs, and recovery work.	Execute approved changes; emergency action under incident authority.	Technical recovery evidence, privileged access, and migration records.
IT Manager / service authority	Risk register, major incident coordination, release oversight. [REQUIRES PROJECT TEAM CONFIRMATION: name accountable IT Manager or equivalent].	Approves major changes, emergency access, restoration, and service communications.	Service continuity and closure of corrective actions.

Governance role	Owns / maintains	Approves / decides	Accountability
Privacy Contact / DPO	Privacy requests, privacy assessments, notice and retention coordination. [REQUIRES PROJECT TEAM CONFIRMATION: identify contact and organization].	Advises or decides privacy response within authority; coordinates any required notification.	Transparent, documented privacy handling.
Developer / technical contributor	Source code and migrations under version control.	No routine production access; contributes reviewed changes.	Tests, review responses, and accurate technical documentation.

12. Decision rights and segregation of duties

Table 11. Decision and approval matrix

Decision	Prepares	Approves	Evidence
Create or change a user/store assignment	Admin	Authorized store owner / manager	Named user, role, store, approver, date, expiry where relevant.
High-impact stock correction	Inventory staff or Admin	Second authorized approver under local threshold	Count, reason, before/after, supporting reference.
Normal code or database change	Developer	Reviewer and release owner	Issue, pull request, tests, migration, release record.
Emergency technical change	System Administrator	IT Manager or delegated incident commander	Incident ID, reason, scope, rollback, time, retrospective review.
Restore or backup-retention change	System Administrator	IT Manager / System Owner	Restore scope, approvals, reconciliation, retention evidence.
Privacy request or disclosure	Privacy Contact with system support	Privacy authority under organization policy	Request, identity verification, decision, disclosure/correction/deletion evidence.

Where project staffing is too small for full separation, the System Owner records the conflict, appoints an independent reviewer for high-risk actions, and retains evidence. No single person should silently request, approve, implement, and close their own high-impact production change.

13. Access governance lifecycle

1. Request: document the user, business duty, role, store scope, start date, duration, and any privileged need.
2. Approve: Admin or designated owner validates identity and need; an independent approver reviews high-risk or cross-store access.
3. Provision: use the managed user function and invite flow selected for production; record the resulting role/store mapping.
4. Review: at least quarterly, compare active users, roles, and store assignments with current duties; record decisions and exceptions.
5. Revoke or change: remove access on separation, role change, compromise, or end of assignment; preserve transaction attribution and reconcile pending offline work.
6. Privileged access: restrict to named administrators, use MFA where enforced, approve and time-bound support access, and review after use.

Production invite path: the repository UI currently calls the generate-invite Edge Function. Confirm the target project deployment, authorization policy, and administrator recovery path: [REQUIRES PROJECT TEAM CONFIRMATION: name production invite owner and supported recovery procedure].

14. Data and privacy governance

The deploying organization determines why personal information is collected, provides the required notice, assigns a Privacy Contact, approves retention, handles rights requests, and assesses incidents. Kodigo can contain names, email addresses, roles, store assignments, supplier contact details, sales and stock activity, payment method and limited reference, audit events, error context, exports, and device/browser details when linked to a user.

Kodigo must not store full card numbers, PINs, one-time passwords, or wallet passwords. Ordinary cash checkout does not require customer identity; collect customer details only where a defined and approved feature requires them.

Privacy handling should be implemented and reviewed in accordance with Republic Act No. 10173, the Data Privacy Act of 2012, its Implementing Rules and Regulations, and applicable National Privacy Commission issuances. This documentation is not a claim that Kodigo is legally certified or formally compliant.

Table 12. Privacy governance fields to complete

Required field	Project-team action
Responsible organization	[REQUIRES PROJECT TEAM CONFIRMATION: insert deploying organization legal name].
Privacy Contact / DPO	[REQUIRES PROJECT TEAM CONFIRMATION: insert name/title and authorized privacy contact].
Official contact details	[REQUIRES PROJECT TEAM CONFIRMATION: insert official email and business address].
Retention schedule	[REQUIRES PROJECT TEAM CONFIRMATION: approve retention periods by record class and legal/business need].
Incident channel and notification authority	[REQUIRES PROJECT TEAM CONFIRMATION: approve support channel, response owner, and external notification decision process].

15. Risk management

Table 13. Initial project risk register

Risk and evidence	Potential effect	Control / owner	Open status
Deployed database may not match repository migration set; several new migrations and dated files exist.	Policies or columns expected by UI may be missing or different.	System Administrator compares applied ledger, policies, functions, grants and triggers before release.	Deployment state requires confirmation.
Offline browser queues have limited conflict handling.	Duplicate, rejected, or stale changes may affect sales and stock.	Admin reconciles pending work; system owner defines offline eligibility; test queue replay and exceptions.	Resolve error items; do not clear browser data.
Privileged operations depend on Edge Function secrets and deployment.	User administration or invite flow may fail or be misconfigured.	Restrict server secrets; verify function auth and role checks; use one documented production path.	Confirm deployment and secret ownership.
MFA enforcement may differ by role or Supabase project settings.	Sensitive accounts could have inconsistent protection.	Confirm policy and required roles; test TOTP and step-up flows.	Deployment and policy confirmation.
Backup plan is described in a runbook but actual subscription, schedule, and restore test are unverified.	Recovery may be unavailable or incomplete.	System Administrator verifies daily backup, 30-day restricted retention baseline, and restore drill; IT Manager approves.	Attach evidence before production acceptance.
Policy owner, service channel, legal identity, and budget are not in repository evidence.	No clear accountability or resourcing for support and privacy response.	System Owner assigns accountable contacts, cost center, and response coverage.	Project team confirmation required.
Generated screenshots and reports may expose store, user, or sales information.	Unnecessary disclosure in academic or public distribution.	Review/redact test data where needed, restrict distribution, and retain only approved evidence.	Review before publication.

16. Change, release, and configuration management

Table 14. Repository and release standards

Area	Required standard
Source of truth	Latest applied SQL migration ledger first; then current runtime code and Edge Functions; then architecture and maintainer documents; baseline schema is not final RBAC state.
Database changes	Add a new ordered migration; do not rewrite an already shared/applied migration. Review policy impact, data migration, rollback, and documentation.

Area	Required standard
Naming and structure	Use descriptive migration filenames; snake_case database identifiers; keep feature UI in the existing components/pages/stores/lib folders; update role types, routes, navigation, RLS, validators and docs together.
Environment values	Browser values use VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY only. Never place service-role secrets in VITE variables, source control, or browser bundles.
Review and testing	Use a reviewed change request; run npm ci, typecheck, lint and build; run integration tests only against an isolated test project with explicit opt-in.
Deployment	Verify target environment and migrations, deploy required Edge Functions, inspect secrets, run smoke tests, and record release/rollback references. Vercel config is present; actual production setup requires confirmation.
Rollback	Prefer frontend/function rollback for code issues; for schema/data damage, restore a verified backup to an isolated replacement and validate before promotion.

17. Incident management and escalation

The ITIL-aligned workflow logs, categorizes, assesses impact and urgency, assigns priority, diagnoses, escalates when needed, validates recovery, and records lessons. Tier 0 and Tier 1 handle self-check and store impact; Tier 2 investigates technical faults; Tier 3 coordinates major, privacy, vendor, or emergency decisions.

Figure 3. ITIL-Aligned Incident Management and Escalation Process

Incident management and escalation workflow

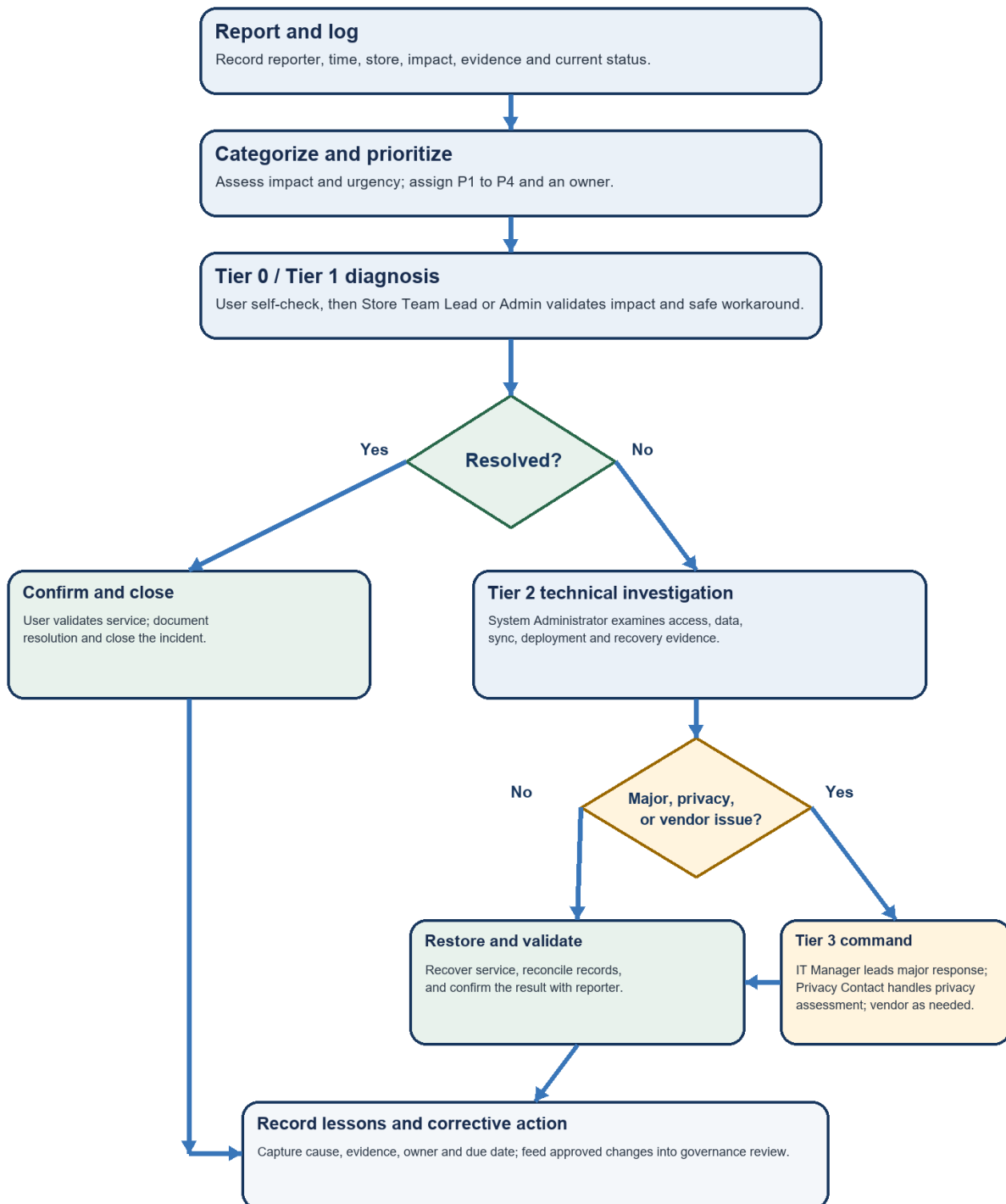


Table 15. Proposed response targets for capstone operations

Tier / priority	Trigger	Target / action	Owner
Tier 0	Single-user how-to or known warning; no security or data concern.	Immediate self-check; up to 10 minutes.	User.
Tier 1	Persistent issue, cash discrepancy, one terminal/store, or access request.	Acknowledge P1/P2 within 15 minutes; P3 within one business hour; aim for workaround within one hour.	Store Team Lead / Admin.
Tier 2	Multi-user issue, failed sync, suspected database/access defect, checkout outage, incorrect data.	Acknowledge within 30 minutes; P1 restore/workaround target 4 hours; P2 one business day.	System Administrator.

Tier / priority	Trigger	Target / action	Owner
Tier 3	Major outage, privacy/security event, restore/emergency change, vendor/legal impact.	P1 acknowledge within 15 minutes; update stakeholders every 60 minutes; incident commander sets recovery target.	IT Manager; Privacy Contact for privacy.
P1 / P2 / P3 / P4	Priority reflects service and data impact; P4 is a request or planned change.	P1 immediate escalation; P2 validate/escalate quickly; P3 standard queue; P4 change/service-request process.	Assigned incident owner.

These values are carried forward as proposed capstone operating targets from the prior manual. The organization must approve service hours, measurement rules, holidays, communication channel, and actual owners before adopting them. They are not contractual service-level agreements.

18. Backup, recovery, audit, and monitoring responsibilities

Table 16. Operational ownership

Control activity	Responsible	Review / approval	Evidence
Daily production backup and restricted retention	System Administrator / provider operator.	IT Manager or System Owner.	Provider backup status, retention configuration, access list.
Pre-migration backup and deployment	System Administrator.	Release approver.	Backup reference, migration log, test evidence.
Restore rehearsal	System Administrator.	IT Manager verifies reconciliation and acceptance.	Restore test project, row-count checks, smoke-test record.
Audit and error review	System Administrator and Admin within assigned scope.	IT Manager for elevated or major cases; Privacy Contact for personal data cases.	Dated review notes, incident links, corrective actions.
Rollback / incident record	System Administrator or incident lead.	IT Manager / incident commander.	Timeline, impact, cause, evidence, recovery and due dates.

The production operations runbook recommends daily Supabase backups, a manual backup before schema deployment, weekly exports of business-critical tables, restricted storage, at least 30 days of backup retention, and restore validation in a separate environment. These are the documented baseline recommendations; the deploying organization must verify that its subscription and configuration actually provide them.

19. User feedback and post-launch improvement

The repository does not confirm a dedicated in-app feedback module. The System Owner should establish an approved support or feedback channel and maintain a feedback log with issue, role/store affected, frequency, business impact, evidence, decision, owner, and due date.

- Collect user feedback during onboarding, support incidents, and a scheduled post-launch review.
- Group feedback into usability, reliability, data accuracy, access, and privacy themes; protect personal data in the record.
- Prioritize changes by operational harm, user frequency, legal/privacy impact, and implementation effort.
- Approve a change, assign an owner and due date, test in an isolated environment, and publish the release note or user instruction update.
- After release, ask the reporting users whether the change solved the problem and record any remaining issue.

Feedback channel and review cadence: [REQUIRES PROJECT TEAM CONFIRMATION: name the support/feedback tool, accountable reviewer, and meeting schedule].

20. Documentation ownership, training, and review

This document set is version 1.2 dated 5 October 2026. The System Owner appoints a document owner who keeps each policy, manual, and design record aligned with the application, migration chain, and approved operating procedures. Changes are reviewed with code and policy owners before release.

- Review at least annually and after material changes to roles, privacy processing, sale lifecycle, inventory controls, hosting, or incident ownership.
- Use version history in the project repository; record change summary, author/reviewer, approval, effective date, and replaced version.
- Keep controlled templates for access requests, risk reviews, incident records, restore tests, release approval, and user acknowledgment.
- Train Admins and Cashiers on sales and cash controls; Inventory staff on units, counts, and adjustments; privileged staff on access, MFA, privacy, and recovery.
- Retain attendance and practical competency evidence in an organization-approved restricted location.

21. Feasibility, schedule, and resource plan

The repository documents a working application, frontend build tools, Supabase integrations, and optional hardware integration. It does not include an approved capstone calendar, labor estimate, hosting budget, subscription tier, device count, or support staffing commitment. The staged plan below is a sequence of work gates; the project team must enter owners, dates, and cost values.

Table 17. Delivery milestones requiring team scheduling

Milestone	Completion evidence	Owner / target date
1. Confirm scope, owner, environment, and migration ledger	Approved scope, deployed-project inventory, role matrix and privacy contacts.	[REQUIRES PROJECT TEAM CONFIRMATION: assign owner and date].
2. Complete staging hardening and acceptance tests	Role/store tests, POS and offline evidence, restore drill, risk disposition.	[REQUIRES PROJECT TEAM CONFIRMATION: assign owner, environment, and date].
3. Train users and approve policies	Signed AUP/BYOD records, training attendance and competency checks.	[REQUIRES PROJECT TEAM CONFIRMATION: set user count, trainer and date].
4. Controlled release and operational handover	Release approval, backup evidence, support route, rollback and incident roster.	[REQUIRES PROJECT TEAM CONFIRMATION: assign approver and date].
5. Post-launch review	Feedback themes, metrics, incidents and prioritized improvement actions.	[REQUIRES PROJECT TEAM CONFIRMATION: set review date and cadence].

Table 18. Resource and budget inventory

Resource	Known project evidence	Cost / quantity to confirm
Software and development	React/Vite source, repository, npm toolchain, TypeScript, browser test environment.	[REQUIRES PROJECT TEAM CONFIRMATION: confirm labor roles/hours and any paid developer tools].
Cloud services	Supabase Auth/Postgres/Edge Functions and Vercel SPA rewrite configuration are used or referenced by code/config.	[REQUIRES PROJECT TEAM CONFIRMATION: identify active provider plans, monthly cost, database/storage/egress assumptions].
Devices and peripherals	Modern browser workstation; optional barcode scanner, printer, and Web Serial cash drawer.	[REQUIRES PROJECT TEAM CONFIRMATION: list device count, existing vs purchased items, and unit/total cost].
Training and support	Role-based user onboarding and incident handling are required by this governance plan.	[REQUIRES PROJECT TEAM CONFIRMATION: assign trainer/support coverage and effort].
Backup and recovery	Operations runbook recommends restricted backup storage and restore tests.	[REQUIRES PROJECT TEAM CONFIRMATION: confirm plan cost, retention, storage size, and restore labor].

22. Course concept integration and evidence

Table 19. Course concepts applied to Kodigo

Concept area	Application in this design and governance	Evidence / section
Documentation ownership and version control	Named owner, version/date, controlled templates, scheduled review, and repository history.	Document Control; Sections 16 and 20.
Security principles and CIA	Least privilege, store scoping, named accounts, confidentiality of exports, integrity of sale/movement records, availability through queueing and recovery.	Sections 8, 9, 13 and 18.
Preventive, detective, corrective control model	Controls are explicitly classified and tied to account, record, and service risks.	Section 9.
User feedback and improvement loop	Collect, triage, approve, implement, release, and re-check user feedback after launch.	Section 19.
Compliance and privacy responsibility	Data Privacy Act framework, responsible organization, privacy contact, rights requests, breach escalation, and no unsupported certification claim.	Section 14.
Evidence-based proposal and call to action	Repository facts, measurable proposed acceptance tests, unresolved assumptions, and approvals requested are explicit.	Sections 2, 10, 21 and 24.

23. Rubric traceability

Table 20. Peer review rubric criterion to document sections

Rubric criterion	Documents / sections carrying the requirement	Original assessment	Status after revision
1. Clarity and Completeness	System Design and Governance 1–2; User Manual 1–2; document controls in all four files.	Partial: purpose and users appeared; measurable outcomes and delivery ownership were incomplete.	Addressed; approval, user-count, and date placeholders remain.
2. System Design and Requirements	System Design and Governance 3–10; Figures 1–2; acceptance measures; user-facing flows in User Manual.	Partial: diagrams and modules existed, but technical architecture, current features and environment state were stale/incomplete.	Addressed from repository evidence; deployed environment and migration ledger need verification.
3. Governance, Security and Compliance	System Design and Governance 11–20; Acceptable Use Policy 3–11; BYOD Policy 2–9.	Partial: RBAC, incidents and privacy notice existed; organizational authority, control classification, and enforcement detail were incomplete.	Addressed; responsible contacts, legal notice, backup configuration and service coverage need approval.
4. Course Concept Integration	System Design and Governance 17, 19–20, 22; AUP 4–10; BYOD 3–8.	Partial: COBIT/ITIL/CIA appeared but concepts did not consistently shape control design or post-launch feedback.	Explicitly mapped to controls and governance sections; course-specific module wording should be checked by adviser.
5. Feasibility and Documentation Quality	All four controlled documents; System Design and Governance 21, 24; User Manual procedures and screenshots.	Partial: professional manual existed but combined policy/system content; dates, budget, schedule and technical caveats were absent or stale.	Four separate structured deliverables; costs, staffing, calendar dates and production configuration remain team confirmations.

24. Decisions requested before submission or production use

- Name the deploying organization, System Owner, AUP/BYOD approver, IT service authority, Privacy Contact/DPO, and support channel.
- Compare the target Supabase migration ledger with repository migrations through migration_40 and dated additions; document the applied state and any drift.
- Confirm actual production and staging hosting, domains, Supabase projects, Edge Function deployment, public browser keys, and protected service-role secret ownership.
- Approve role-specific MFA requirements, access-review owner, incident service hours and targets, backup retention, restore approval, and data-retention schedule.

- Provide a dated milestone plan, assigned people/hours, itemized provider/device/training costs, and funding authority.
- Review live screenshots for any personal, supplier, or transaction data before wider academic or public distribution.

Approval request: the capstone adviser and designated System Owner should approve the technical scope, governance owners, proposed acceptance tests, policy drafts, schedule, and budget assumptions; authorize a staging validation cycle; and return the confirmed values to the document owner for incorporation.